Chasing Your Tail NG — Complete Hardware Guide

> **Firmware:** Chasing Your Tail NG (CYT-NG) · **Upstream:** ArgeliusLabs/Chasing-Your-Tail-NG (MIT) **Chip:** ESP32 (generic, 4MB) · **Cyber Controller profile:** `cyt-ng` (esptool backend, merged single-bin @ `0x0`, GitHub-release resolver, not suicide-capable) **This guide:** purchase → build → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

Chasing Your Tail NG is a **counter-surveillance / personal-privacy tool**: it passively watches Wi-Fi (and BLE) device activity over time and place, then flags devices that keep reappearing near you across multiple locations — the signature of something, or someone, following you. It works by analyzing **probe requests** (the network names devices broadcast while hunting for known Wi-Fi) and correlating repeat appearances with GPS, producing reports and map visualizations of suspicious "persistent" devices.

Upstream's reference implementation is a **Linux Python stack built around Kismet** (capture), WiGLE (SSID geolocation), and GPS correlation, emitting Markdown/HTML reports and KML maps. The Cyber Controller `cyt-ng` profile targets an **ESP32 build** of the same idea — a single merged firmware image written with esptool and resolved from the project's GitHub releases — to act as a small, portable probe-request sensor. **Honesty note:** at the time of writing upstream ships the Python tool but has **no tagged GitHub releases and no ESP32 `.bin` asset**, so the flasher's release resolver returns source-only/empty until an official or user-supplied ESP32 image exists (verify: §9). This guide covers both the ESP32 flashing path and the Linux capture stack the analysis actually depends on.

2. Legal & Safety

This is a passive, defensive tool — keep it that way. CYT-NG only *listens* to the probe requests and BLE advertisements that nearby devices broadcast openly; it does not transmit attacks, deauthenticate, spoof, or join networks. That makes it dramatically lower-risk than offensive ESP32 firmware. However, passively capturing Wi-Fi management frames and logging MAC addresses, SSIDs, and locations **can still be regulated** (wiretap/privacy/data-protection law varies widely by jurisdiction). Lawful use here is **personal counter-surveillance and personal safety** — checking whether *you* are being followed in *your* surroundings. Do not build dossiers on identifiable people. Treat captured MACs/SSIDs/GPS as sensitive personal data and store it securely (this is exactly why the tool encrypts WiGLE credentials). Check local law before logging.

3. Hardware & Purchasing

There are two hardware paths; pick by how deep you want to go.

A) ESP32 sensor (what the `cyt-ng` profile flashes): a generic ESP32-WROOM-32, **4MB flash, DIO mode, 80 MHz** (per profile). Cheap, pocketable, serial-only via Cyber Controller.

B) Reference Linux capture stack (full CYT-NG experience): a Linux host plus a monitor-mode Wi-Fi adapter, optional GPS, and Kismet — this is what produces the persistence reports and KML maps.

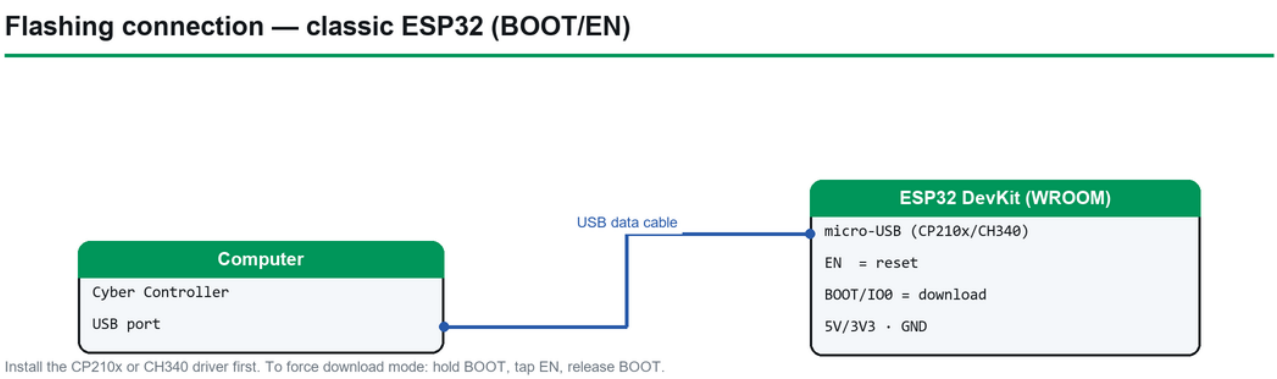
Role	Hardware	Why	Where to buy (search terms)
ESP32 sensor (profile `cyt-ng`)	**ESP32-WROOM-32 DevKit, 4MB**	What the profile flashes; cheap portable probe sensor	AliExpress/Amazon: "ESP32 DevKit V1" / "ESP32-WROOM-32 38-pin"
Capture host	**Linux laptop** or **Raspberry Pi 4/5**	Runs Kismet + the CYT-NG Python stack	Any Linux laptop; Raspberry Pi official resellers
Monitor-mode Wi-Fi	**Alfa AWUS036NHA** (Atheros AR9271, 2.4 GHz) or **AWUS036ACM/ACH** (dual-band)	Reliable monitor mode for Kismet	Amazon/Alfa: "AWUS036NHA" / "AWUS036ACM"
GPS (optional)	**USB NMEA GPS** (GlobalSat BU-353S4) or Bluetooth GPS	Location correlation → stalking/persistence detection	Amazon: "BU-353S4 USB GPS"

Accessories: a quality **data** USB cable (not charge-only) for the ESP32; optional microSD on the host for capture storage. **Avoid** ESP32 clones with the wrong flash size — verify **4MB+**. **Verify** that your chosen Wi-Fi chipset supports monitor mode under your kernel/Kismet version before buying.

4. Building / Assembly

- **ESP32 sensor:** pre-built DevKit — no assembly, just a data cable. Install the USB-serial driver: **CP210x** (most DevKits) or **CH340** (cheap clones). Without it no COM port appears.
- **ESP32 firmware availability:** there is currently **no published upstream ESP32 binary** (verify: §9). If/when an official ESP32 build is released you flash it (§5); otherwise the working device *is* the Linux stack below, with the ESP32 used as a portable serial sensor once a build exists.
- **Linux stack build:** install **Kismet** (distro package or the kismetwireless apt repo); put the adapter into monitor mode (upstream ships `./start_kismet_clean.sh`); then `pip3 install -r requirements.txt`, run `python3 migrate_credentials.py` to encrypt WiGLE credentials, and edit **`config.json`** (`kismet_db_path` glob, `log_directory`, `ignore_lists_directory`, `time_windows`). Install **pandoc** if you want HTML reports.
- **Per-host gotcha:** confirm the adapter actually *enters* monitor mode (chipset/driver dependent) and that Kismet is writing a `.kismet` SQLite DB whose path matches `config.json` — the analyzers read from it.

5. Flashing & First Run (via Cyber Controller)



How to connect the board to flash it (classic ESP32 — BOOT/EN download mode).

1. Connect the ESP32 by USB; open Cyber Controller → **Flash** tab.
2. **Port:** pick the board's COM/tty (click *Refresh* if missing → driver issue).
3. **Firmware Profile:** `Chasing Your Tail NG` (profile `cyt-ng`; backend **esptool**, chip **esp32**).
4. Click **Flash.** The profile's **GitHub-release resolver** queries the upstream releases API for a `.bin` asset and writes it as a **single merged image at offset `0x0`** (4MB / DIO / 80 MHz, **115200** baud).
5. **If no release asset is found** — currently the case (verify: §9) — the resolver returns *source-only/empty*. Either **point the flasher at your own local merged ESP32 `.bin`** or wait for an official release. After a successful flash, first boot streams a banner / scan output on serial at 115200.

6. Integrate into Cyber Controller

- **Profile:** `cyt-ng` — esptool backend, `image_model: merged-single-bin`, `app_offset: 0x0`, resolver `github_release` (asset regex `^(.+)\.bin$`, chip fixed to `esp32`, label `CYT-NG {env}`, `on_error: source_only_empty`). **`supports_suicide: false`** — no self-destruct for this profile. **`protocol: null`** → generic serial: output appears as **raw text** in the terminal, not auto-parsed.
- **Control:** open the **Devices** tab → select the port → set **Firmware = Chasing Your Tail NG** → **Connect** at **115200**. Read the sensor's detection stream in the persistent terminal.
- **Targets/Cross-Comm:** because `protocol` is null, this profile is **not** wired into protocol-aware Targets-pool population or auto-routing the way Marauder is (verify exact behavior in your build) — treat it as a passive sensor/monitor rather than a target source.
- **Backup first:** use **Backup** in the Flash tab to dump existing flash before overwriting.

7. Usage (end-to-end)

ESP32 sensor mode: Devices tab → Connect @115200 → watch probe/BLE detections stream in the terminal. Leave it running as you move between locations; the same device MAC reappearing across separate places is the "being followed" signal you're looking for.

Full Linux reference workflow (where the analysis lives): 1. **Capture:** `./start_kismet_clean.sh` — puts the adapter in monitor mode; Kismet logs to a `.kismet` DB. 2. **Live monitor:** `python3 chasing_your_tail.py` (or GUI `python3 cyt_gui.py`) — buckets probe requests into recent/medium/old time windows. 3. **Analyze history:** `python3 probe_analyzer.py --days 7` (add `--wigle` for SSID geolocation, `--all-logs`). 4. **Surveillance detection:** `python3 surveillance_analyzer.py --kismet-db <path>` (or `--demo`); refine with `--stalking-only --min-persistence 0.8`; export with `--output-json analysis_results.json`. GPS clustering is handled by `gps_tracker.py`. 5. **Review output:** Markdown reports in `surveillance_reports/` (HTML via pandoc), KML maps in `kml_files/` (open in Google Earth). Devices that persist across multiple locations are the ones to investigate. Add your own devices to `ignore_lists/` to suppress noise.

8. Troubleshooting

- **ESP32 no COM port:** install CP210x/CH340 driver; use a data cable; on Linux add your user to `dialout`.
- **Flash returns empty / "no release asset":** upstream publishes no ESP32 `.bin` (verify: §9) — supply a local merged bin, or skip the ESP32 path and run the Linux stack.
- **Flash "Failed to connect":** hold **BOOT** during connect, lower the flash baud in Settings, and close any serial monitor holding the port.
- **Boot-loop / brick: Erase Flash** then re-flash; verify the board is genuinely 4MB+.
- **Adapter won't enter monitor mode:** wrong chipset/driver — verify Kismet supports it on your kernel.
- **Kismet DB not found:** `config.json` `kismet_db_path` glob must match the actual `.kismet` file location.
- **WiGLE errors:** run `python3 migrate_credentials.py` and confirm encrypted creds; WiGLE is optional.
- **No HTML reports:** install **pandoc**.
- **Empty surveillance results:** capture longer and across multiple locations; lower `--min-persistence`.

9. Sources

- Upstream: https://github.com/ArgeliusLabs/Chasing-Your-Tail-NG (README, `CLAUDE.md`, scripts; **MIT**). Reference stack is Python + Kismet + WiGLE + GPS, not a microcontroller port.
- Releases: https://github.com/ArgeliusLabs/Chasing-Your-Tail-NG/releases — **no releases / no ESP32 `.bin` asset published at time of writing** (verify before relying on the flasher resolver).
- Cyber Controller profile: `src/config/profiles/cyt_ng.json` (esptool, esp32, 4MB/DIO/80 MHz, baud 115200, merged-single-bin @ `0x0`, `github_release` resolver, `supports_suicide: false`, `protocol: null`).
- Capture deps: Kismet (https://www.kismetwireless.net), WiGLE (https://wigle.net).
- Hardware: ESP32-WROOM-32 DevKit (4MB); monitor-mode adapters (Alfa AWUS036-series); USB GPS (BU-353S4). Verify chipset/kernel monitor-mode support and current prices at purchase time; vendor links change.